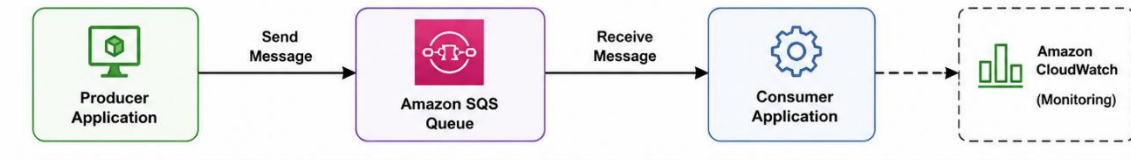
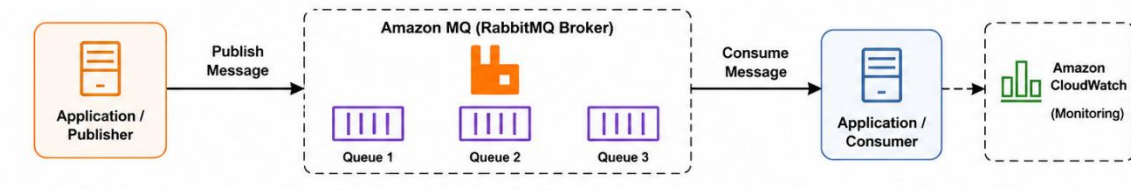


Challenge-06

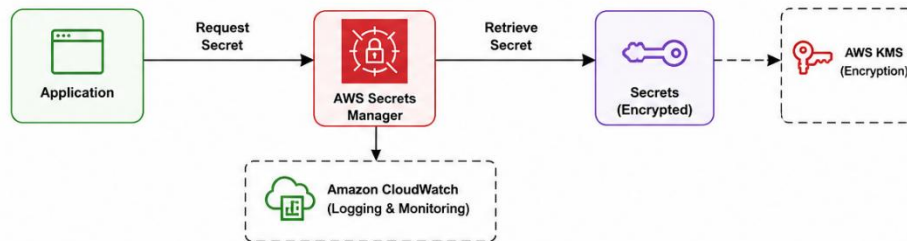
1. AWS SQS Architecture



2. Amazon MQ (RabbitMQ) Architecture



3. AWS Secrets Manager Architecture



WS SQS (Simple Queue Service)

Objective

Implement a message queuing service to decouple applications and enable asynchronous communication.

What is AWS SQS?

AWS SQS is a fully managed message queue service that allows applications to communicate without being tightly connected.

Use Cases

- Order processing systems
- Background job processing
- Microservices communication
- Decoupling application components

Architecture

Producer Application

|

v

SQS Queue

|

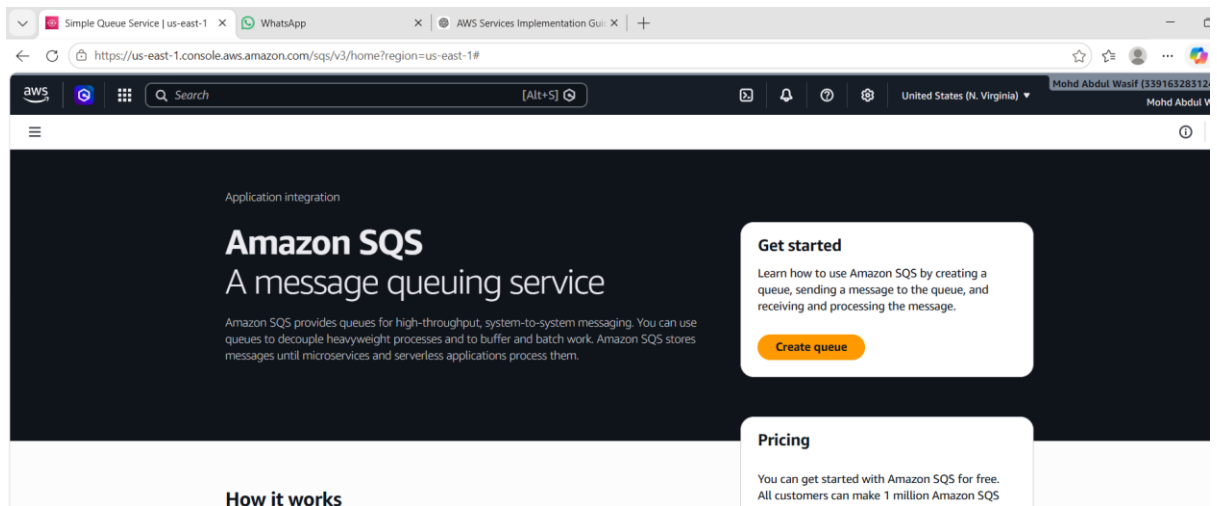
v

Consumer Application

Hands-on Implementation

Step 1: Create SQS Queue

1. Login to AWS Console.



2. Navigate to SQS.
3. Click Create Queue.
4. Select Standard Queue.

5. Name: demo-queue.

Create queue

Details

Type
Choose the queue type for your application or cloud infrastructure.

☒ **Standard** [Info](#)
At-least-once delivery, message ordering isn't preserved

- At-least once delivery
- Best-effort ordering

☐ **FIFO** [Info](#)
First-in-first-out delivery, message ordering is preserved

- First-in-first-out delivery
- Exactly-once processing

You can't change the queue type after you create a queue.

Name

demo-queue

A queue name is case-sensitive and can have up to 80 characters. You can use alphanumeric characters, hyphens (-), and underscores (_).

Configuration [Info](#)
Set the maximum message size, visibility to other consumers, and message retention.

Visibility timeout [Info](#)
30 Seconds

Message retention period [Info](#)
4 Days

6. Create Queue.

Queue demo-queue created successfully
You can now send and receive messages.

demo-queue [Edit](#) [Delete](#) [Purge](#) [Send and receive messages](#) [Start DLQ redrive](#)

Details [Info](#)

Name demo-queue	Type Standard	ARN arn:aws:sqs:us-east-1:339163283124:demo-queue
Encryption Amazon SQS key (SSE-SQS)	URL https://sqs.us-east-1.amazonaws.com/339163283124/demo-queue	Dead-letter queue -

[More](#)

[Queue policies](#) [Monitoring](#) [SNS subscriptions](#) [Lambda triggers](#) [EventBridge Pipes](#) [Dead-letter queue](#) [Tagging](#) [Encryption](#) [Dead-letter queue redrive](#)

Step 2: Send Messages

1. Open queue.
2. Click Send and Receive Messages.
3. Enter message:

Hello Khadeer

Send and receive messages
Use this page to send, retrieve and view messages, helping you experiment with various queue features.

Send message [Info](#)

Message body
Enter the message to send to the queue.

Message group ID - optional, new [Info](#)
A group identifier for the message to allow fair processing across message groups in a standard queue.

Delivery delay [Info](#)
The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.
 Seconds
Should be between 0 seconds and 15 minutes.

► **Message attributes - optional** [Info](#)

[Clear content](#) [Send message](#)

4. Send message.

Step 3: Receive Messages

1. Click Poll for Messages.

Receive messages [Info](#)

[Edit poll settings](#) [Stop polling](#) [Poll for messages](#)

Messages available 1 **Polling duration** 30 **Maximum message count** 10 **Polling progress** Success 0 receives/second

Messages (1)

☐	ID	Sent	Size	Receive count
☐	1373ad62-ebf8-4330-a797-9dfb30b501bc	2026-06-15T12:06+05:30	11 bytes	1

2. View received message.

Received message: 1373ad62-ebf8-4330-a797-9dfb30b501bc

[Body](#) [Attributes](#) [Details](#)

[Done](#)

3. Delete message after processing.

Observations

- Messages are stored reliably.
- Applications can communicate asynchronously.
- Queue handles traffic spikes efficiently.

What I Learned

- Difference between synchronous and asynchronous communication.
 - Queue-based architecture.
 - Message retention and visibility timeout concepts.
-

Amazon MQ

Objective

Deploy a managed message broker using Amazon MQ.

What is Amazon MQ?

Amazon MQ is a managed message broker service that supports:

- ActiveMQ
- RabbitMQ

It helps migrate existing messaging applications to AWS without changing application code.

Use Cases

- Enterprise messaging systems
- Legacy applications
- Banking applications
- Event-driven architectures

Architecture

Application A

|

v

Amazon MQ

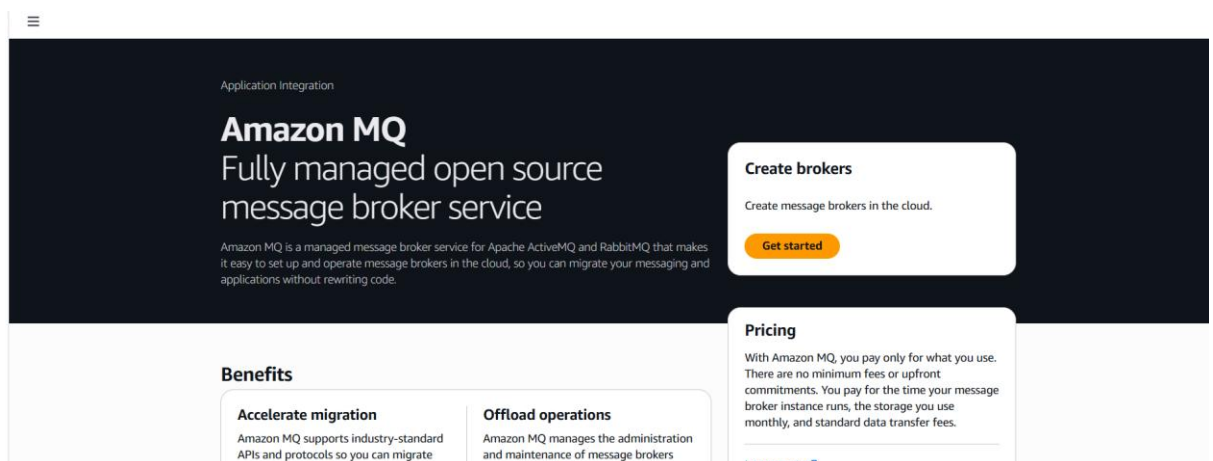
|
v

Application B

Hands-on Implementation

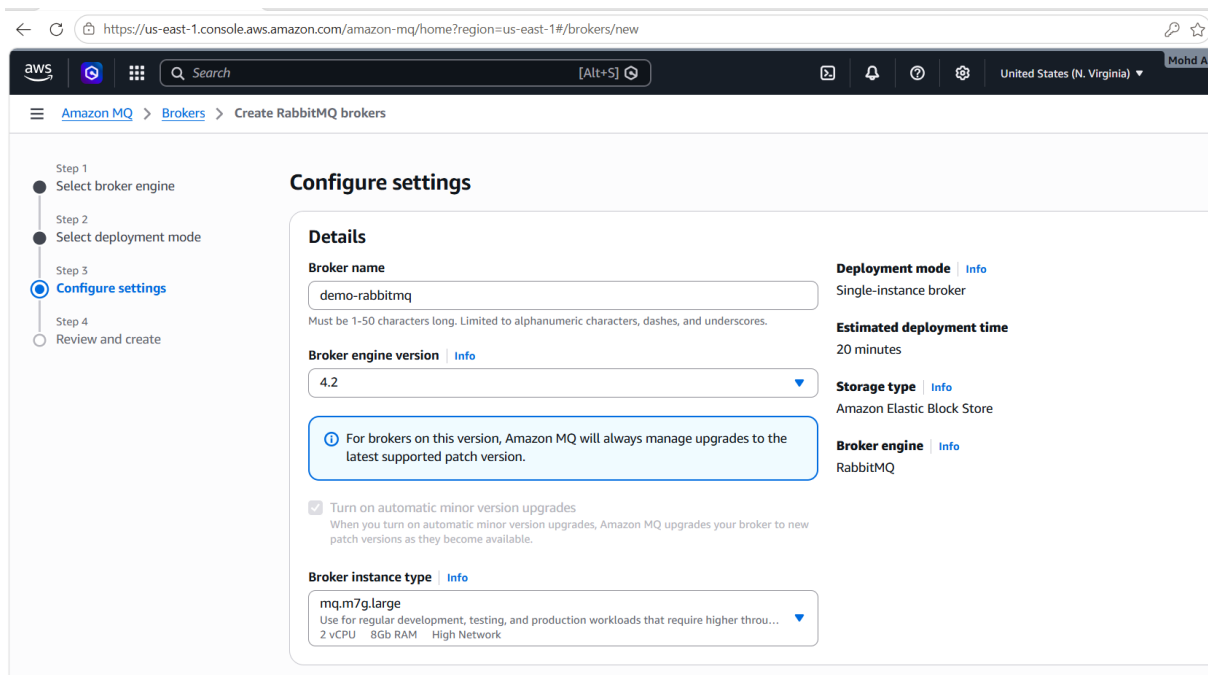
Step 1: Create Broker

1. Open Amazon MQ Console.
2. Click Create Broker.



3. Select RabbitMQ.
4. Broker Name:

demo-rabbitmq



5. Deployment Mode:

Single Instance

6. Create username/password.

RabbitMQ access

Username
Create a user with administrator permissions. This user can create additional users using the RabbitMQ management API or web console.

Can't contain commas (,), colons (:), equals signs (=), or spaces.

Password

Minimum 12 characters, at least 4 unique characters. Can't contain commas (,), colons (:), equals signs (=), spaces or non-printable ASCII characters.

☒ Show password

Additional access methods - [new info](#)
Authenticate and authorize users using the broker configuration. Supported methods include OAuth 2.0, IAM, LDAP, HTTP, SSL and mutual TLS for broker i

☐ Enable additional access methods

► **Additional settings**

7. Launch broker.

Step 2: Access Broker

1. Wait until status becomes Running.

The screenshot shows the AWS Management Console for the 'us-east-1' region. The left sidebar shows 'Amazon MQ' > 'Brokers'. The main content area is titled 'Brokers for us-east-1 (1)'. It contains a table with one broker:

Name	Status	Creation time (Local)	Broker engine	Deployment mode	Instance type	Data replication role
demo-rabbitmq	Running	June 15, 2026 at 12:14 (UTC+5:30)	RabbitMQ	Single-instance broker	mq.m7g.large	-

Below the table, a message states: 'demo-rabbitmq is selected.'

2. Open RabbitMQ Web Console.

The screenshot shows the RabbitMQ Management Web Console login page. The URL in the browser is 'https://b-ccac68e9-7406-44d2-b20e-760189cb8424.mq.us-east-1.on.aws'. The page features the RabbitMQ logo and a login form with the following fields:

- Username:
- Password:
- A 'Login' button.

3. Login with configured credentials.

Overview

Totals

Queued messages [last minute](#) ?

Currently idle

Message rates [last minute](#) ?

Currently idle

Global counts ?

Connections: 0 Channels: 0 Exchanges: 8 Queues: 0 Consumers: 0

Nodes

Name	File descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats	+/-
rabbit@localhost	51 500000 available	505 1048576 available	129 MiB 4.3 GiB high watermark	187 GiB 10.0 GiB low watermark	13m 34s	2	basic 8 rss	This node All nodes	

Churn statistics

Ports and contexts

Export definitions

Import definitions

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Step 3: Create Queue

1. Navigate to Queues.
2. Create queue:

test-queue

Queues

All queues (0)

Pagination

Page 1 of 0 - Filter: [] ☐ Regex ?

... no queues ...

Add a new queue

Type: Default for virtual host

Virtual host: /

Name: test-queue

Durability: Durable

Arguments: [] = [] String

Add [Auto expire](#) ? [Message TTL](#) ? [Overflow behaviour](#) ? [Single active consumer](#) ? [Dead letter exchange](#) ? [Dead letter routing key](#) ? [Max length](#) ? [Max length bytes](#) ? [Maximum priority](#) ? [Leader locator](#) ?

Add queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Use these settings:

- **Type:** Default for virtual host
- **Virtual host:** /
- **Name:** test-queue
- **Durability:** Durable
- **Arguments:** Leave empty

Then click **Add queue**.

3. Publish test messages.

Pagination

Page of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	test-queue	quorum	D Args	running	0	0	0				

▼ Add a new queue

Type:

Send a Test Message

1. Click **test-queue**.
2. Scroll down to **Publish message**.
3. Enter:

Payload

```
{
  "message": "Hello Amazon MQ"
}
```

▼ Publish message

Message will be published to the default exchange with routing key **test-queue**, routing it to this queue.

Headers: ?

=

String ▼

Properties: ?

=

Payload:

```
{
  "message": "Hello Amazon MQ"
}
```

Payload encoding:

String (default) ▼

Publish message

4. Click **Publish Message**.

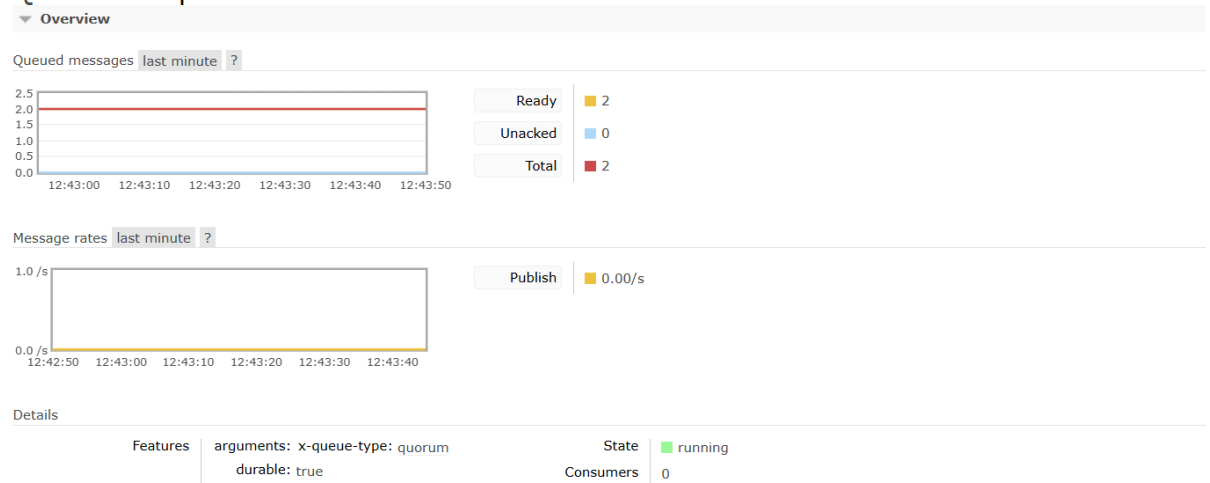
Verify Message

After publishing, you should see:

Ready: 1

Total: 1

Queue test-queue



Consume Message

1. In the same queue page, find **Get messages**.

2. Message count:

1

3. Click **Get Message(s)**.

The message should appear below.

Warning: getting messages from a queue is a destructive action. ?

Ack Mode:

Encoding: ?

Messages:

Get Message(s)

Message 1

The server reported 1 messages remaining.

Exchange	(AMQP default)
Routing Key	test-queue
Redelivered	0
Properties	delivery_mode: 2 headers:
Payload 35 bytes Encoding: string	{ "message": "Hello Amazon MQ" }

Observations

- Managed RabbitMQ administration.
- Easy broker setup.
- Built-in monitoring and security.

What I Learned

- Message broker concepts.
- Difference between Queue and Broker.
- RabbitMQ management in AWS.

AWS Secrets Manager

Objective

Securely store and retrieve application secrets.

What is AWS Secrets Manager?

AWS Secrets Manager helps store and manage sensitive information such as:

- Database passwords
- API Keys
- Tokens
- Application credentials

Use Cases

- RDS database credentials
- Third-party API keys
- Application secrets
- Automated secret rotation

Architecture

Application

|

v

AWS Secrets Manager

|

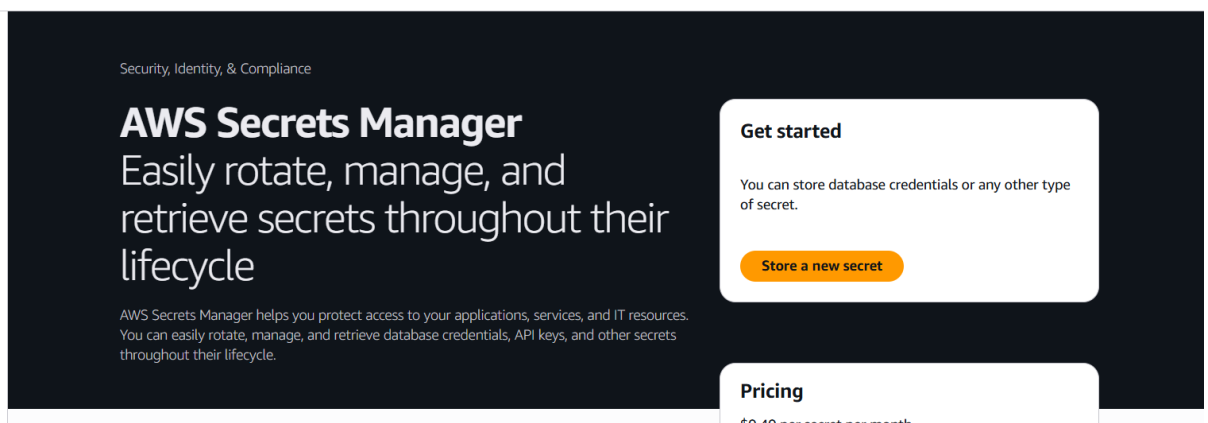
v

Encrypted Secret

Hands-on Implementation

Step 1: Create Secret

1. Open Secrets Manager.



2. Click Store a New Secret.
3. Select:

Other type of secret

- 4. Add key-value pair:

Value

Key

Step 1

Choose secret type

Step 2

Configure secret

Step 3 - optional

Configure rotation

Step 4

Review

Choose secret type

Secret type

☐ Credentials for Amazon RDS database

☐ Credentials for Amazon DocumentDB database

☐ Credentials f

☐ Credentials for other database

☐ Managed external secret
Secrets vended by your third-party software vendor.

☒ Other type o
API key, OAuth

Key/value pairs

Key/value

Plaintext

Wasif

wa@WL20271187

+ Add row

usernae

Khadeer

password
dwa@WL20271187

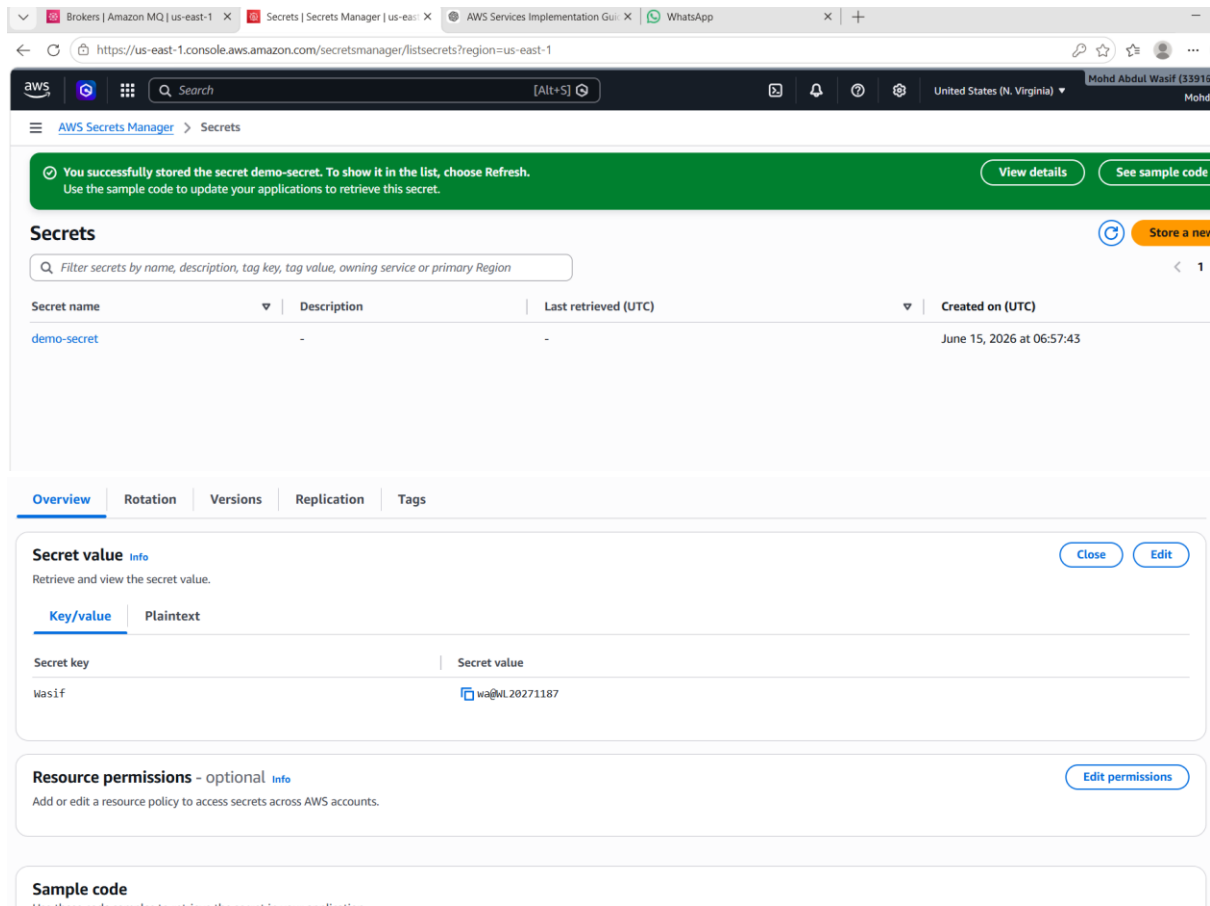
- 5. Secret Name:

demo-secret

- 6. Save secret.

Step 2: Retrieve Secret

- 1. Open secret.
- 2. Click Retrieve Secret Value.
- 3. View stored credentials.



Observations

- Secrets are encrypted automatically.
- Secure alternative to storing passwords in code.
- Centralized secret management.

What I Learned

- Secret lifecycle management.
- Encryption using KMS.
- Secure credential storage.

Difference Between SQS, Amazon MQ, and Secrets Manager

Feature	SQS	Amazon MQ	Secrets Manager
Purpose	Message Queue	Message Broker	Secret Storage
Managed Service	Yes	Yes	Yes

Feature	SQS	Amazon MQ	Secrets Manager
Stores Messages	Yes	Yes	No
Stores Passwords	No	No	Yes
Supports Protocols	AWS API	AMQP, MQTT, STOMP, JMS	AWS API
Primary Use	Decoupling Apps	Enterprise Messaging	Credential Management

Conclusion

Through this hands-on exercise, I gained practical experience with AWS messaging and security services. I successfully created and tested AWS SQS queues, deployed an Amazon MQ RabbitMQ broker, and securely stored credentials using AWS Secrets Manager. These services improve application scalability, reliability, security, and maintainability in cloud environments.